Project Plan and Progress

Project: E-Commerce & Order Management System (Olist) · **Group:** TarushTanishkTanishq
Team: Tarush Banke (project lead) · Tanishk Varshney · Tanishq Mahajan · **Status date:** 15 October 2026

---

1. Process model and team organisation

Model: Agile Scrum with 4 short sprints. We picked Scrum over Waterfall because the high-level requirements (course rubric) were fixed, but the details depended on what we found in the data. For example, the ML task changed from *product recommendation* to *late-delivery prediction* once profiling showed that only 3.1% of customers order twice.

Scrum role	Member
Product owner (represents course requirements)	Tarush Banke
Scrum master (runs stand-ups, tracks the board)	Tarush Banke
Development team	Tarush Banke, Tanishk Varshney, Tanishq Mahajan

Team structure: each member owns one layer (database, back end, front end + ML), and all three review and test each other's work.

Communication: online stand-up on most days (about 15 min), a group chat for quick questions, and a sprint review + retrospective call at the end of each sprint.

Tools: Git + GitHub (version control), a shared task board (backlog → in progress → review → done), VS Code, PostgreSQL 16 / psql, Python, Mermaid (diagrams).

---

2. Work breakdown structure

```
1 E-Commerce & Order Management System
├── 1.1 Project management
│   ├── 1.1.1 Planning, sprint planning, task board          (Tarush)
│   ├── 1.1.2 Risk register, meeting minutes                 (Tarush)
│   └── 1.1.3 Mid-semester report and demo video             (All)
├── 1.2 Requirements
│   ├── 1.2.1 Dataset profiling                              (Tarush, Tanishq)
│   └── 1.2.2 SRS: functional / non-functional requirements  (Tarush)
├── 1.3 Database design & implementation
│   ├── 1.3.1 ER/EER model, relational schema                (Tarush)
│   ├── 1.3.2 Keys, FDs, normalization to BCNF               (Tarush)
│   ├── 1.3.3 Staging load, cleaning, transform              (Tarush)
│   ├── 1.3.4 Indexes and views                              (Tarush)
│   ├── 1.3.5 Triggers and stored procedures                 (Tanishk)
│   ├── 1.3.6 Security: roles, grants, RLS                   (Tanishk)
│   └── 1.3.7 Transactions and concurrency demos             (Tanishk)
├── 1.4 Application
│   ├── 1.4.1 DB access layer + Flask API                    (Tanishk)
│   ├── 1.4.2 Report queries Q1–Q16                          (Tanishk)
│   ├── 1.4.3 Front end: CRUD, search, reports, lab          (Tanishq)
│   └── 1.4.4 ER model & schema page                         (Tanishq)
├── 1.5 AI/ML
│   ├── 1.5.1 Problem framing, feature view in SQL           (Tanishq, Tarush)
│   ├── 1.5.2 Training, model selection, evaluation          (Tanishq)
│   └── 1.5.3 Integration with DB and UI                     (Tanishq, Tanishk)
└── 1.6 Testing & documentation
    ├── 1.6.1 Test plan and test execution                   (Tanishq)
    ├── 1.6.2 Design document, README                        (Tarush)
    └── 1.6.3 Screenshots / evidence                         (Tanishq)
```

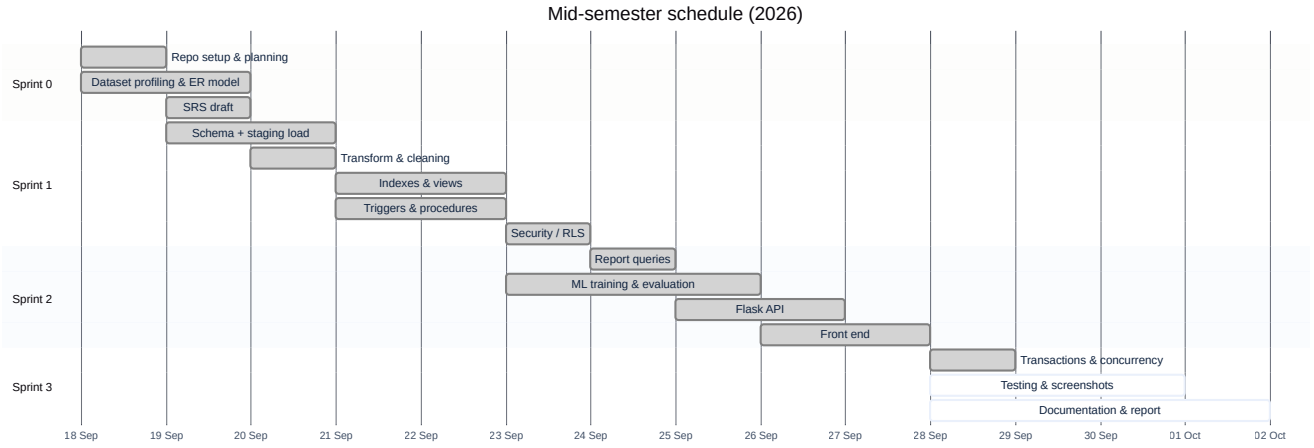
---

3. Schedule

3.1 Sprint plan

Sprint	Dates	Goal	Deliverables	Status
0	18–19 Sep	Understand the problem, set up the project	Repo, dataset profile, ER model v1, SRS draft	Done
1	20–23 Sep	Working database with all rules	Schema, load + cleaning, indexes, views, triggers, procedures, security	Done
2	24–27 Sep	Usable application + ML model	Flask API, front end, reports Q1–Q16, trained model	Done
3	28 Sep–1 Oct	Hardening and release	Transaction/concurrency demos, testing, docs, mid-sem report	Done (test execution and submission package completed 15 Oct)

3.2 Gantt chart



---

4. Effort estimation

Basic COCOMO (organic mode) on the delivered code, excluding the bundled Mermaid library:

Item	Value
Size	≈ 4.9 KLOC (SQL 1.7k, Python 2.0k, JS/CSS/HTML 1.2k)
Effort E = 2.4 × KLOC^1.05	≈ 12.8 person-months
Duration D = 2.5 × E^0.38	≈ 6.6 months
Average staff E / D	≈ 1.9 people

Actual: 3 people × ~2 weeks, i.e. about 1.5 person-months.

Why the gap: COCOMO is calibrated on industrial projects that include full requirements, formal reviews, documentation, deployment and maintenance, whereas this is a scoped academic release. SQL is also very dense: one line can express a join over five tables, and PostgreSQL does most of the work (constraints, RLS, window functions). We take the COCOMO figure as an upper bound, and it is one reason we cut some features from the scope (see section 1.2 of the SRS and "Out of scope" in the README).

---

5. Risk management

ID	Risk	Probability	Impact	Mitigation	Status
R1	Dataset quality problems (duplicates, missing zips, untranslated categories) break loading	High	High	Load into a text-only staging schema first; clean in SQL; expose problems in `v_data_quality`	Closed: 278 + 7 missing zips and 2 categories handled
R2	Chosen ML task not learnable from the data	Medium	High	Profile data before choosing; keep a simple baseline; honest metrics	Occurred: switched recommendation → late-delivery prediction
R3	Data leakage makes ML metrics look better than they are	Medium	High	Time-based split; seller history uses only past deliveries; delivery dates never used as features	Controlled

ID	Risk	Probability	Impact	Mitigation	Status
R4	Team members blocked by each other (API needs schema, UI needs API)	Medium	Medium	Schema frozen at the end of Sprint 1; API contract agreed before UI work	Controlled
R5	Security mistakes (SQL injection, over-privileged app user)	Low	High	Bound parameters only; `olist_app` has no privileges of its own; RLS tested with the seller role	Controlled
R6	Environment differences (Windows paths, PostgreSQL not on PATH)	High	Medium	`.bat` scripts check prerequisites; README troubleshooting table	Occurred once (PostgreSQL service missing on one laptop); fixed with a check in `run_app.bat`
R7	Schedule slip before the mid-semester deadline	Medium	High	Must-have vs nice-to-have list; nice-to-haves (LightGBM, what-if prediction) done last	Open, on track
R8	Loss of work	Low	High	Everything in Git, pushed daily	Controlled
R9	Secrets committed to the repository	Low	Medium	`.gitignore` for `.env`; only documented demo passwords used	Controlled

---

6. Configuration management

- **Repository:** single `main` branch, small commits, each commit authored by the member who did the work.
- **Not versioned:** the dataset CSVs (licence + size), the trained model file (reproducible with `train_model.bat`), `__pycache__`, `.env`.
- **Reproducibility:** `setup_db.bat` → `train_model.bat` → `run_app.bat` rebuilds the whole system from the CSVs.
- **Versioning:** `v1.0` = mid-semester release.

7. Current progress (status on 15 Oct 2026)

Work package	Planned	Status	Evidence
Requirements (SRS, use cases)	Sprint 0	Done	`02_Requirements/`
ER model, schema, keys, FDs, BCNF	Sprint 0–1	Done	`03_Design/Database_Design.pdf`
Data load + cleaning (≈1.6 M rows)	Sprint 1	Done	TC-01, TC-02, TC-05
Indexes, views, triggers, procedures	Sprint 1	Done	TC-03 – TC-14, index demo
Security: roles, grants, RLS	Sprint 1	Done	TC-15 – TC-21
Flask API + front end (CRUD, search, reports, lab, ER page)	Sprint 2	Done	TC-22 – TC-28
ML model + DB/UI integration	Sprint 2	Done	TC-29 – TC-31
Test plan written	Sprint 3	Done (28 Sep)	`docs/TestPlan.md`
Test execution with recorded results	Sprint 3	Done late (15 Oct, planned by 1 Oct)	`05_Testing/`

Work package	Planned	Status	Evidence
Mid-semester report, demo video	Sprint 3	Done	`01_Report/`, `07_Demo/`

Overall: all must-have features of the mid-semester release work end to end (31/31 test cases passed, 2 with a dataset-version note).

8. Delayed or pending tasks

Task	Original date	Status / reason	New date
Execute the test plan and record actual results	1 Oct	Test cases were written on 28 Sep but not executed before the sprint closed	Done 15 Oct
Re-run TC-02 and TC-24 on the current Kaggle files	–	Test run used an older public copy of the dataset (reviews 100,000 vs 99,224 rows)	Before demo
LightGBM in the model comparison	Sprint 2 (nice-to-have)	Optional package not installed on the test machine; model selection ran without it	Second half

9. Weekly progress record

Week	Dates	Main work completed	Commits (Tarush / Tanishk / Tanishq)
1	18–24 Sep	Repo set-up; SRS; ER model; full schema; staging load and transform; indexes; views; triggers; procedures; security; first ML pipeline; report queries Q1–Q16	18 / 13 / 6
2	25 Sep–1 Oct	DB access layer and Flask API; complete front end; ML service; transactions/concurrency demos; DDL/DML, index and normalization demos; test plan and screenshots; design doc, project plan, README; fixes D-01, D-02	7 / 11 / 12
3	2–8 Oct	No commits recorded in Git for this week	–
4	9–15 Oct	Test execution (31 test cases), automated test scripts, mid-semester report	–